

A02 – Mothership Mayhem

Welcome aboard captain! Your team at UNC Deep Space Systems wants to build a modular mothership capable of swapping power sources on the fly, firing thrusters, and running science experiments. You'll build tiny modules that communicate cleanly with proper encapsulation. You'll use dependency injection to refuel or change your power source mid-mission without touching mothership code!

How the Classes Work Together

<code>AModule.java</code>	# Abstract base class for all modules
<code>ExperimentModule.java</code>	# Runs and reports science experiments
<code>FuelGenerator.java</code>	# Fuel-powered implementation of <code>IPowerGenerator</code>
<code>IModule.java</code>	# Interface that requires the status update method
<code>IPowerGenerator.java</code>	# Interface for any power generator
<code>Main.java</code>	# The entry point
<code>Mothership.java</code>	# Orchestrates modules
<code>SolarGenerator.java</code>	# Solar-powered implementation of <code>IPowerGenerator</code>
<code>ThrusterModule.java</code>	# Consumes power to provide thrust

The **Mothership** coordinates a collection of **AModules**—most notably a **ThrusterModule**, an **ExperimentModule**, and one injected **IPowerGenerator**. The power generator can be either a **SolarGenerator** or a **FuelGenerator**, both of which implement the **IPowerGenerator** interface.

Inheritance and Interfaces

- **AModule** is an abstract base class that defines shared behavior.
- **IPowerGenerator** is an interface that specifies what a power source must do.

Dependency Injection (DI) in Practice

The mothership does not instantiate its power generator directly. Instead, the chosen generator (e.g., `new SolarGenerator()` or `new FuelGenerator(fuel)`) is injected into the mothership.

Apprentice

This stage is where we lay the foundation.

IModule.java

```
public interface IModule {  
    String getName();  
    void statusReport(String moduleStatus, boolean isSuccessful);  
}
```

AModule.java

Create an abstract class AModule that implements IModule. It has:

- Field: `name` (String), passed via constructor.
- Methods:
 - `getName()`
 - `statusReport(String moduleStatus, boolean isSuccessful)`

ThrusterModule.java

Extends AModule.

- Fields: `int fuel = 100`, `boolean lastFired = false`.
 - Methods:
 1. Override `statusReport`.
 2. `boolean thrust(int availablePower)` with fuel and power check.
-

Enchanter: Generators

IPowerGenerator.java

```
public interface IPowerGenerator extends IModule {  
    int generatePower();  
}
```

SolarGenerator.java

- Constructor uses `super`.
- Override `statusReport`: prints “Solar Generator will never die” then calls base.
- `generatePower()` returns 10.

FuelGenerator.java

- Field: `int fuel`, set by constructor.
 - Override `statusReport`: print fuel then call base.
 - `generatePower()` consumes fuel in chunks of 10.
-

Sorcerer: ExperimentModule

- Fields: `experimentName`, `double[] parameters`, `result`, `hasRun`.
 - Methods:
 1. `runExperiment()` iterates parameters, randomizes results.
 2. `getSummary()` reports depending on `hasRun`.
 3. Override `statusReport`.
-

Mothership.java

- Fields: `IPowerGenerator`, `ThrusterModule`, `ExperimentModule`, `ArrayList<IModule>`.
 - Constructor injection to initialize.
 - Methods: `requestPower()`, `fireThruster()`, `runExperiment()`, `printStatusReports()`, `getExperimentSummary()`.
-

Mission Control (Main.java)

1. Choose a power source (Solar or Fuel).
 2. Assemble modules.
 3. Launch mothership with constructor injection.
 4. Request power and print.
 5. Fire thruster.
 6. Run experiment.
 7. Print status reports.
-

Merlin: Extra Credit

For extra credit, add a new module extending `AModule`, override its `statusReport`, update mothership constructor, and call it from `Main`.